Type checking.

* Type checking → Checking that whether each operator receives proper number of arguments of proper data types.

A=B+(*D

* Two types
  - Static type checking → during compile time
  - Dynamic type checking → during run time

* Type checker verifies that the type of a construct matches that as expected.

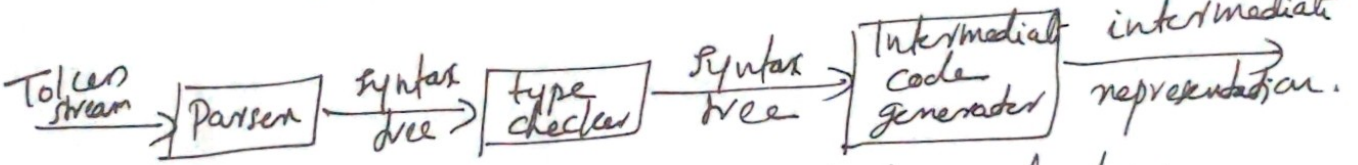

Fig. Position of type checker.

* Type systems → is a collection of rules for assigning type expressions to the various parts of a pgm. A type checker implement a type system.

Specification of a simple type checker

* A type checker is specified for a simple language in which the type of each identifier must be declared before the identifier is used.

Consider nonterminal P, consisting of a sequence of declarations D followed by a single expression E

$P \rightarrow D; E$

$D \rightarrow D; D \mid id : T$

$T \rightarrow char \mid integer \mid array(num)\ of\ T$

$E \rightarrow num \mid id \mid E\ mod\ E \mid E[E]$

Fig. Grammar for source language

* One program generated by the ~~pgm~~ grammar is,

sum : integer ;
sum mod 200

Consider,

$P \rightarrow D : E$
$\rightarrow id : T ; E$
$\rightarrow sum : T ; E$
$\rightarrow sum : integer ; E$
$\rightarrow sum : integer ; E\ mod\ E$
$\rightarrow sum : integer ; id\ mod\ E$
$\rightarrow sum : integer ; sum\ mod\ E$
$\rightarrow sum : integer ; sum\ mod\ num$
$\rightarrow sum : integer ; sum\ mod\ 200$

* In the translation scheme, the action associated with the production $D \rightarrow id : T$ saves a type in a symbol table entry for an identifier. The action addtype (id.entry, T.type) is applied to synthesized

attribute entry pointing to the symbol table entry for id and a type expression represented by synthesized attribute type of nonterminal T.

* The part of a translation scheme that saves the type of an identifier is as follows

$P \rightarrow D ; E$

$D \rightarrow D ; D$

$D \rightarrow id : T$ {addtype (id.entry, T.type)}

$T \rightarrow$ char { T.type = char }

$T \rightarrow$ integer { T.type = integer }

~~$T \rightarrow \uparrow T_1$ { T.type =~~

$T \rightarrow$ array [num] of T_1 { T.type = array [1 .. num.val, T_1.type) }.

Type checking of expressions

* Use a function lookup(e) to fetch the type saved in the symbol-table entry pointed by e. When an identifier appears in an expression, its declared type is fetched and assigned to the attribute type.

✓ $E \rightarrow id$ { E.type = lookup (id.entry) }

Expression formed by applying mod operator is

✓ $E \rightarrow E_1$ mod E_2 { E.type = if E_1.type = integer and E_2.type = integer then integer ~~otherwise~~ else type error }

* In an array reference $E_1[E_2]$, the index expression E_2 must have ~~int~~ type integer

✓ $E \rightarrow E_1[E_2]$ {E.type = if E_2.type = integer and E_1.type = array(s,t) then t else type-error}

index set element type

* Type checking of statements

✓ $S \rightarrow id = E$ {S.type = if id.type = E.type then void else type-error}

✓ $S \rightarrow$ if E then S_1 {S.type = if E.type = boolean then S_1.type else type error}

~~S → t~~ Type checking of Functions.

* The rule for checking the type of a function is,

✓ $E \rightarrow E_1(E_2)$ {E.type = if E_2.type = s and E_1.type = s → t then t else type error}

i/p arg type

the fn type